

יסודות מדעי המחשב

חזרה



דפנה לוי רשתי



משתנים

טיפוסים פרימיטיביים

- **int** מספרים שלמים, בעל טווח מוגבל ($\pm 2,147,483,647$)
- **char** תווים: אותיות, ספרות, סימנים וסימנים מיוחדים. לדוגמה: 'a', ' ', '\n'
- **float** מספרים עם נקודה עשרונית. לדוגמה: 32.24, 1.0, 23232.454
- **boolean** משתנה בוליאני, כלומר - משתנה המסוגל להכיל "אמת" או "שקר" בלבד
- **long** דומה ל- int, אך בעל טווח גדול הרבה יותר
- **double** דומה ל- float, אך בעל טווח ודיוק גדולים יותר
- **byte** - **short** שני סוגים שימושיים פחות, הדומים ל- int אך בעלי טווח קטן יותר.
- **String** מחרוזת, רצף של תווים – אינו פרימיטיבי, נרחיב בהמשך

משתנים

- **- שם חוקי** – רצוי שם משמעותי. שם חוקי יכול להכיל אותיות (אנגליות, גדולות או קטנות), ספרות, סימני קו-תחתון (`_`) וסימני `$`. השם לא יכול להתחיל בספרה, אבל יכול להתחיל בכל אחד מהסימנים האחרים. כמו כן, השם אינו יכול להכיל סימן שלא הוזכר כאן כמו `'^'`, `'%'` או `'*'`. כמו כן, השם לא יכול להיות מילה שמורה של Java

• - הצהרה

- `int a;`
- `int a, b, c;`

• - השמה

- `a = 5;`
- `int sum = 8;`
- `int x, y = 5;`

הוראת הקלט למשתנים מטיפוסים שונים

```
Scanner input = new Scanner(System.in);
```

טיפוס המשתנה	דוגמה לקלט
--------------	------------

int	num = input.nextInt();
-----	------------------------

long	bigNum = input.nextLong();
------	----------------------------

double	avg = input.nextDouble();
--------	---------------------------

char	avg = input.next().charAt[0];
------	-------------------------------

String	schoolName = input.next();
--------	----------------------------

עצם מטיפוס
Scanner
עדיף עצם
סטטי שיוכר
בכל המחלקה.

שימו לב שאין
כאן את
הטיפוס

פלט – כתיבה למסך ה-Console

`System.out.println();` // ירידת שורה

`System.out.println(String);` // הדפסה וירידת שורה

`System.out.print(String);` // הדפסה

(call toString())

```
01. package test;
02. public class Test
03. {
04.     public static void main(String args[])
05.     {
06.         int x = 5;
07.         int y = 10;
08.         System.out.println(x+y);
09.         System.out.println("x+y");
10.         System.out.println("" + x + y);
11.         System.out.println("5" + "10");
12.         System.out.println(5 + 10 + x + y);
13.         System.out.println("output " + (x + y));
14.         System.out.println("output " + x + y);
15.     }
16. }
```



```
run:
15
x+y
510
510
30
output 15
output 510
```

פלט – כתיבה למסך ה-Console



תבנית עיצוב

```
System.out.format("the number is %d \n" , num);  
System.out.format("the first number is %d and the second number is %d ", x, y);
```

שירשור

```
System.out.print("the number is" + num);  
System.out.print("the first number is " + x + " and the second number is " + y);
```

אופרטורים חשבוניים

דוגמה	משמעות	אופרטור
<code>int sum = 5+6; // sum=11</code>	חיבור	+
<code>double dif = 2.5-1.2; // dif=1.3</code>	חיסור	-
<code>double price = 1.23*2; // price=2.46</code>	כפל	*
<code>double avg = 26/5; // avg=2.46</code>	חילוק	/
<code>int digit = 21%4; // digit = 1</code>	שארית חלוקה בשלמים	%

שארית



מחזוריות



אריזה בגדלים שונים



ספרות מספר



זוגיות

פעולות אריתמטיות על טיפוסים שונים

op = {+,-,*,/}	תוצאה	הערות
int op int	int	החלק השלם (מנה) int/int שארית int%int
double op double	double	מספר ממשי עשרוני double/double
int op double double op int	double	

אופרטור	כתיבה מקוצרת	כתיבה לא מקוצרת
++	num++	num+1
--	num--	num-1
+=	num+=2 num+=num2	num=num+2 num=num+num2
-=	num-=3.5 num-=num2	num=num-3.5 num=num-num2
=	num=5 num*=num2	num=num*5 num=num*num2
/=	num/=6 num/=num2	num=num/6 num=num/num2
%=	num%=2 num%=num2	num=num%2 num=num%num2



זהירות!
הסדר משמעותי.
+=
הוא בעל משמעות שונה
כך גם ++a מול a++

I'm not lazy. I'm just in
energy-saving mode.

(int)(double)
(int)(double*int)
(int)(double/double)

(double)(int) / int
(double)(int / int)

(char)int
(int)char

לכל תו יש מספר
בהתאם ל
Unicode
'A' = 65
(char)(int)'A' + 1
→ 'B'

גם אני נגד טיפולי המרה

ברביעי בבוקר אוסרים על הטירוף
הזה אחת ולתמיד



פונקציות שכיחות בספריה Math

פונקציה

ערך מוחלט	<code>Math.abs(_)</code>	מחזירה את הערך המוחלט של X . טיפוס הערך בהתאם לטיפוסו של X
חזקה	<code>Math.pow(_ , _)</code>	מחזירה חזקה של a בחזקת b . $a^b \leftarrow$ תוצאת החישוב תמיד מטיפוס <code>double</code>
שורש ריבועי	<code>Math.sqrt(_)</code>	מחזירה את השורש הריבועי של X . תוצאת החישוב תמיד מטיפוס <code>double</code>
עיגול	<code>Math.round (_)</code>	שבר מ-0.5 ומעלה מעביר לשלם הבא. תוצאת החישוב תמיד מטיפוס <code>double</code>
מינימום	<code>Math.min(_ , _)</code>	מוחזר הערך המינימאלי בין a ל- b . טיפוס הערך בהתאם לטיפוס של a ו- b אם לפחות אחד מהם מטיפוס <code>double</code> הערך המוחזר הוא מטיפוס <code>double</code>
מקסימום	<code>Math.max(_ , _)</code>	מוחזר הערך המקסימאלי בין a ל- b . טיפוס הערך בהתאם לטיפוס של a ו- b אם לפחות אחד מהם מטיפוס <code>double</code> הערך המוחזר הוא מטיפוס <code>double</code>

מספר אקראי - Random

השלבים שצריך לבצע כדי לקבל מספר אקראי בתחום ערכים מסוים:
• הגדרת משתנה מטיפוס Random, ובניית מופע מטיפוס זה:

```
Random rnd = new Random();
```

• הגרלת מספר אקראי בטווח שהוגדר על ידי הפעלת
הפעולה nextInt (או פעולה אחרת) על העצם.

Life is **RANDOM**
but it's all good cuz
SO AM I

מספר אקראי - Random

- מספר אקראי בתחום שבין 0 ל- x (לא כולל x)

```
int n = rnd.NextInt(x);
```

- מספר אקראי בתחום שבין 0 ל- x (כולל x)

```
n = rnd.NextInt(X+1);
```

- מספר אקראי בתחום שבין x (כולל) ל- y (לא כולל y) כאשר $x < y$

```
n = x + rnd.nextInt(y-x);
```

- מספר אקראי בתחום שבין x (כולל) ל- y (כולל) כאשר $x < y$

```
n = x + rnd.nextInt(y-x+1);
```



ביצוע מותנה



switch

תנאי
מקון

תנאי
מורכב

תנאי
פשוט

משתנה
בוליאני

הוראת תנאי

מאפשרת לנו לבדוק קיומו של תנאי כלשהו :
ערכו של משתנה בוליאני, השוואה בין ערכים של משתנים וכו'

```
if (a < 100) .....  
if (num % 2 == 0) ...  
if (find) System.out.println("YES")  
else System.out.println("NO")
```



סימני יחס בביטוי בוליאני

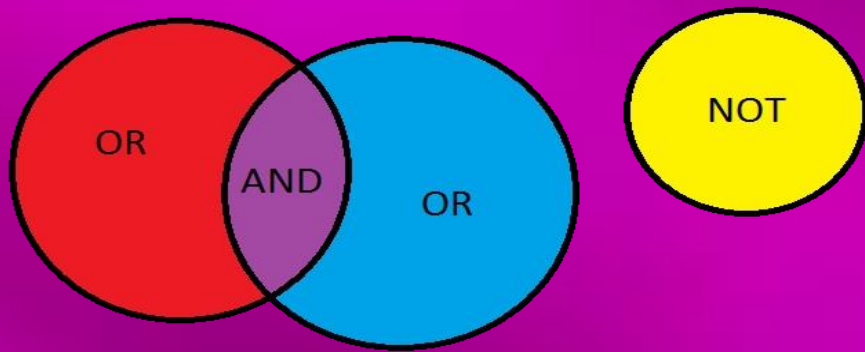
היחס במילים	סימן היחס – כתיבה מתמטית	סימן היחס – כתיבה ב- Java	דוגמה לביטוי בוליאני
שווה	=	==	(a==b)
גדול מ-	>	>	(a>b)
גדול שווה	≥	>=	(a>=b)
קטן מ-	<	<	(a<b)
קטן שווה	≤	<=	(a<=b)
שונה	≠	!=	(a!=b)

```
if (x > 0) {  
    System.out.println("x is positive");  
}
```

```
if (x % 2 == 0) {  
    System.out.println("x is even");  
} else {  
    System.out.println("x is odd");  
}  
}
```





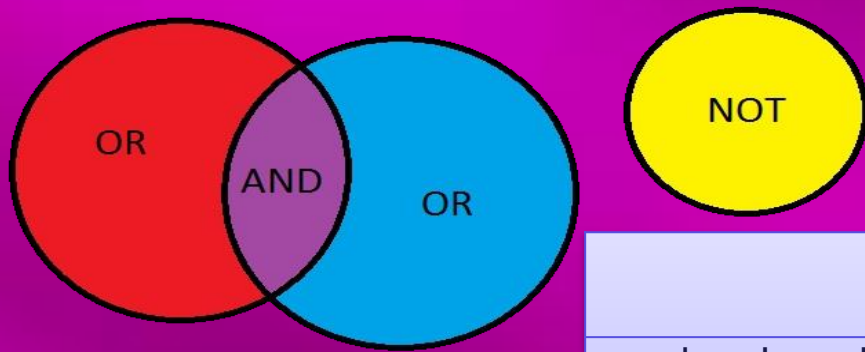


תנאי מורכב

```
if ((num > 9) && (num < 100)) //and
if ((num % 2 == 0) || (num % 3 != 0)) //or
```

a	b	a && b
True	True	True
True	False	False
False	True	False
False	False	False

a	b	a b
True	True	True
True	False	True
False	True	True
False	False	False



תנאי מורכב

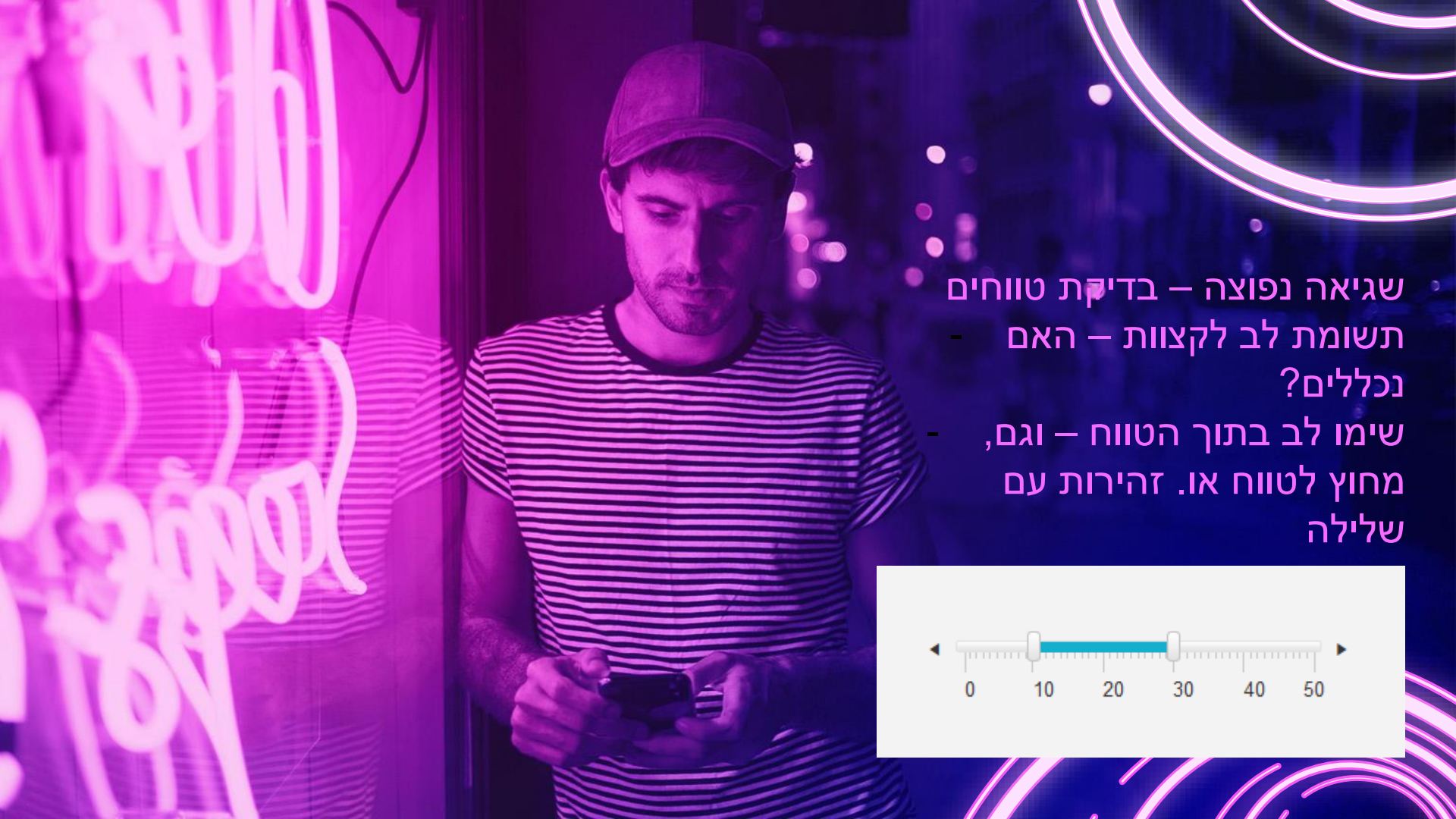
שם	סימן ב- java	דוגמאות
לא	!	אם a לא גדול מ- b ... אם a שונה מ- b (אם a לא שווה ל- b) ... <code>if (!(a > b))...</code> <code>if (!(a == b)) ...</code>
וגם	&&	אם a גדול מ- b וגם a גדול מ- c ... <code>if ((a > b) && (a > c)) ...</code>
או		אם a שונה מ- b או a לא גדול מ- c ... <code>if ((a != b) !(a > c)) .</code> <code>..</code>

גדול אינו ההיפך מקטן!

נא להוסיף סוגריים מתאימים!

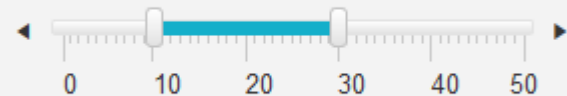
- `!(A && B)` is the same as `!A || !B`
- `!(A || B)` is the same as `!A && !B`
- `!(x < 5 && y == 3)` is the same as `x >= 5 || y != 3`
- `!(x >= 1 || y != 7)` is the same as `x < 1 && y == 7`





שגיאה נפוצה – בדיקת טווחים

- תשומת לב לקצוות – האם נכללים?
- שימו לב בתוך הטווח – וגם, מחוץ לטווח או. זהירות עם שלילה



```
if (a>=b)
    if (a>=c)
        System.out.println (a);
    else
        System.out.println (c);
else
    if (b>=c)
        System.out.println (b);
    else
        System.out.println (c);
```



```
if (x > 0) {  
    System.out.println("x is positive");  
} else if (x < 0) {  
    System.out.println("x is negative");  
} else {  
    System.out.println("x is zero");  
}
```



```
switch (expression) {  
    case constant1:  
        sequence-statements-of-1  
        break;  
    case constant2:  
        sequence-statements-of-2  
        break;  
    ...  
    case constantN:  
        sequence-statements-of-N  
        break;  
    default:  
        default-sequence-statements  
        break;  
}
```

ערכי ה case חייבים להיות
קבועים, ומטיפוס המשתנה.
טיפוס המשתנה יכול להיות
שלם, תו, מחרוזת או טיפוס
מנייה enum
ה break הוא רשות,
ובהעדרו, תמשיך בדיקת שאר
המקרים
אין חובה ל default
ניתן להשאיר case ללא
הוראות לביצוע
גם כאן מותר קיבון

משתנה בוליאני

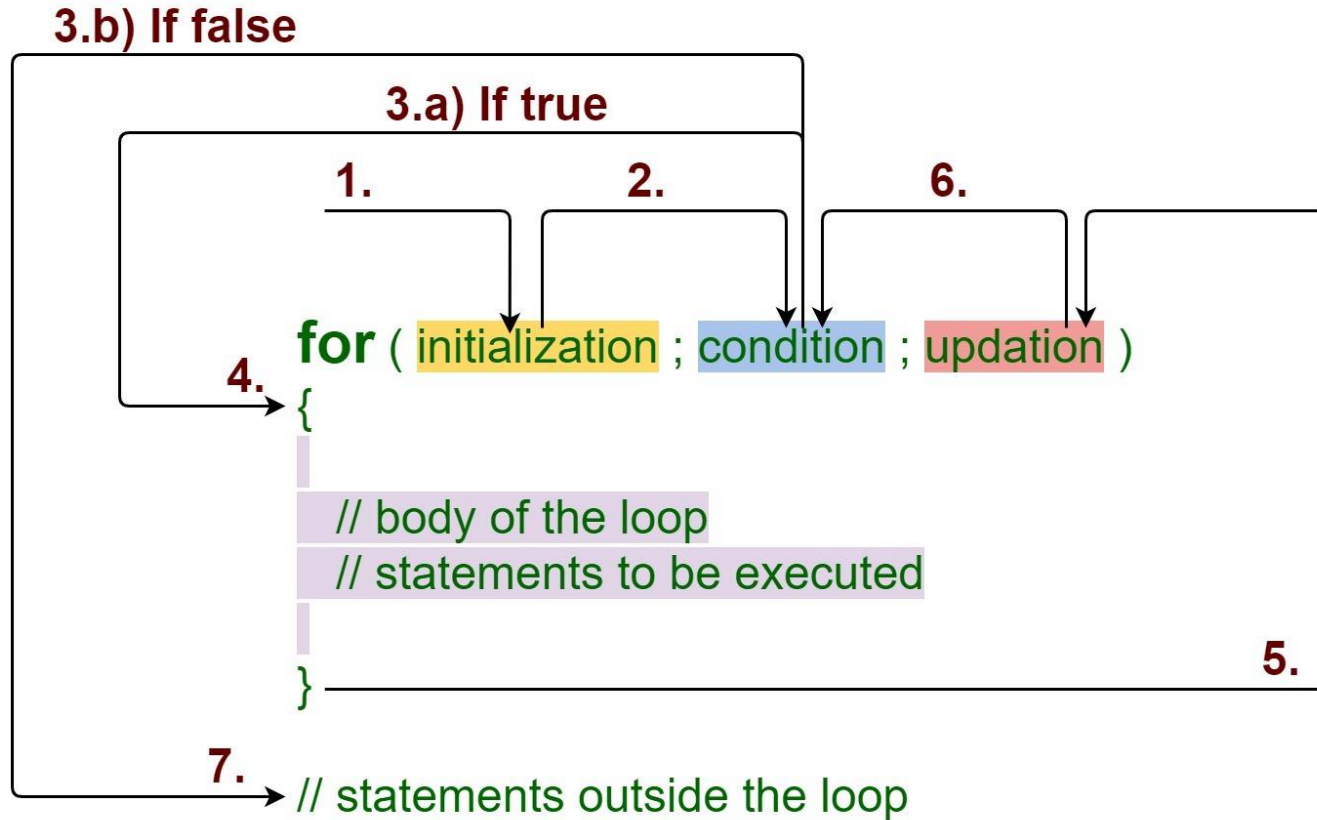
```
boolean flag;  
flag = true;  
boolean testResult = false;  
  
boolean evenFlag = (n % 2 == 0); // true if n is even  
boolean positiveFlag = (x > 0); // true if x is positive
```

```
if (evenFlag) {  
    System.out.println("n was even when I checked it");  
}  
  
if (!evenFlag) {  
    System.out.println("n was odd when I checked it");  
}
```



ביצוע חוזר

For Loop



```

1 public class ForLoops{
2
3     public static void main(String []args){
4
5         for (int i = 0 ; i < 10; i+=3){
6             System.out.println("i " + i);
7         }
8
9         int k;
10        for (k = 0 ; k < 3; k++){
11            System.out.println("in");
12        }
13        System.out.println("k " + k);
14
15        int sum;
16        for (k = 0, sum = 0 ; k < 10; k+=3, sum+=50);
17        System.out.println("sum " + sum);
18
19        int count = 500;
20        for (int i = 0 ; i < 10 && count < 600; i++, count+=50){
21            System.out.println("count " + count);
22        }
23        System.out.println("count " + count);
24
25        /*
26        for ( ; ; ){
27            System.out.println("?");
28        }
29        */
30
31    }
32 }
33

```

\$javac ForLoops.java

\$java -Xmx128M -Xms16M ForLoops

```

i 0
i 3
i 6
i 9
in
in
in
k 3
sum 200
count 500
count 550
count 600

```

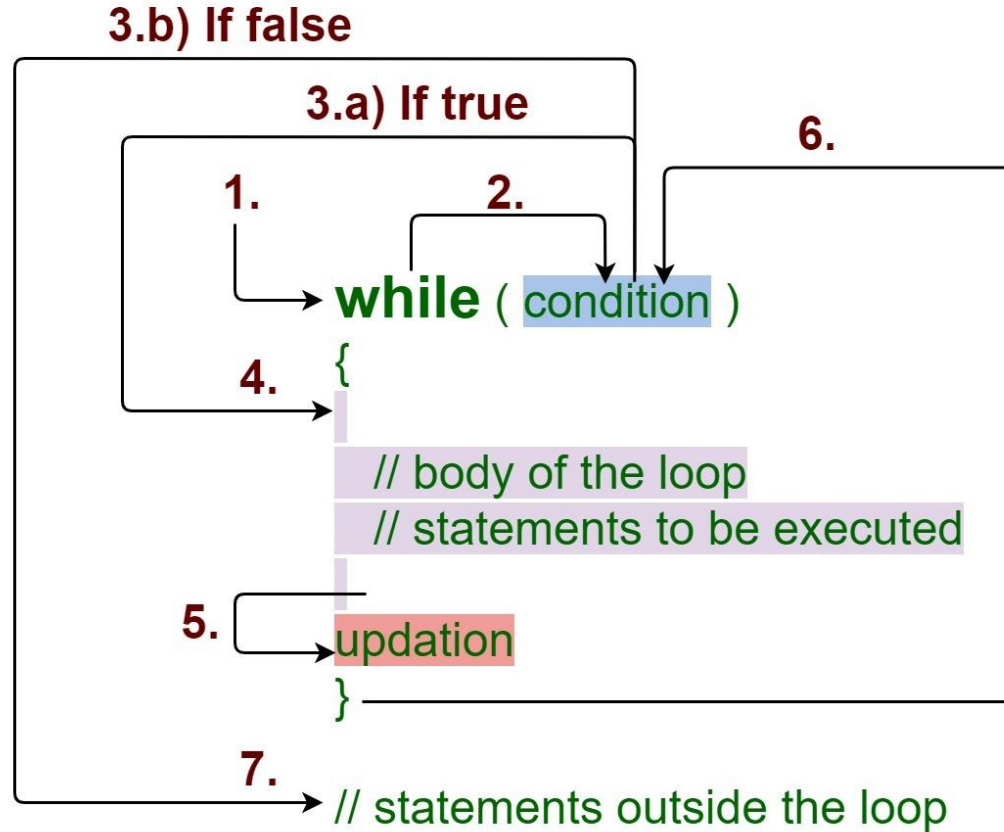
```
1 public class ForLoops{
2
3     public static void main(String []args){
4
5         for (int i = 1 ; i <= 100 ; i++){
6             if (i % 10 == 0)
7                 System.out.println(i);
8         }
9
10        for (int i = 1 ; i <= 10 ; i++){
11            System.out.println(i*10);
12        }
13
14        for (int i = 10 ; i <= 100 ; i+=10){
15            System.out.println(i);
16        }
17
18    }
19 }
```

```
import java.util.Random;
public class Program
{
    public static void main(String[] args)
    {
        boolean is5= false; // הצהרה ואיתחול של הדגל
        int num, dig;
        Random rndNum = new Random();
        num = 1000 + rndNum.nextInt(1000000); // הגרלת מספר
        System.out.print("the number is : " + num);

        while ((!is5) && (num>0)) // התנאי המורכב
        {
            dig = num % 10;
            if (dig == 5) is5=true;
            num = num/10;
        }
        if (is5) System.out.println(" contain the digit 5");
        else System.out.println("does not contain the digit 5");
    }
}
```

אסור להוסיף
אסור לקצר ולכתוב:
is5 = dig==5

While Loop



לולאת זקיף

```
Scanner in = new Scanner(System.in);
int sum = 0, tip;
System.out.print("Enter your first tip for today. " +
                 "End the list of tips with -1: ");
tip = in.nextInt();

while (tip != -1){
    sum = sum + tip;
    System.out.print("Enter the next tip. " +
                     "End the list of tips with -1:");

    tip = in.nextInt();
}

System.out.println("You have earned " + sum + "shekels");
```

לולאת קלט

```
Scanner in = new Scanner(System.in);
System.out.print("Enter 2 digits num: ");
int m = in.nextInt();

while (m <= 9 || m >= 100){
    System.out.print("Enter 2 digits num: ");
    m = in.nextInt();
}

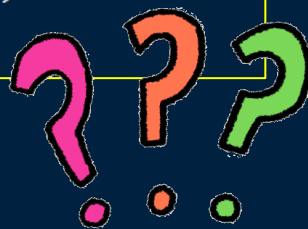
System.out.println(m + "is 2 digits num!");
```

```
Random rnd = new Random();  
int num = 9000 + rnd.nextInt(1000);  
boolean has5 = false;
```

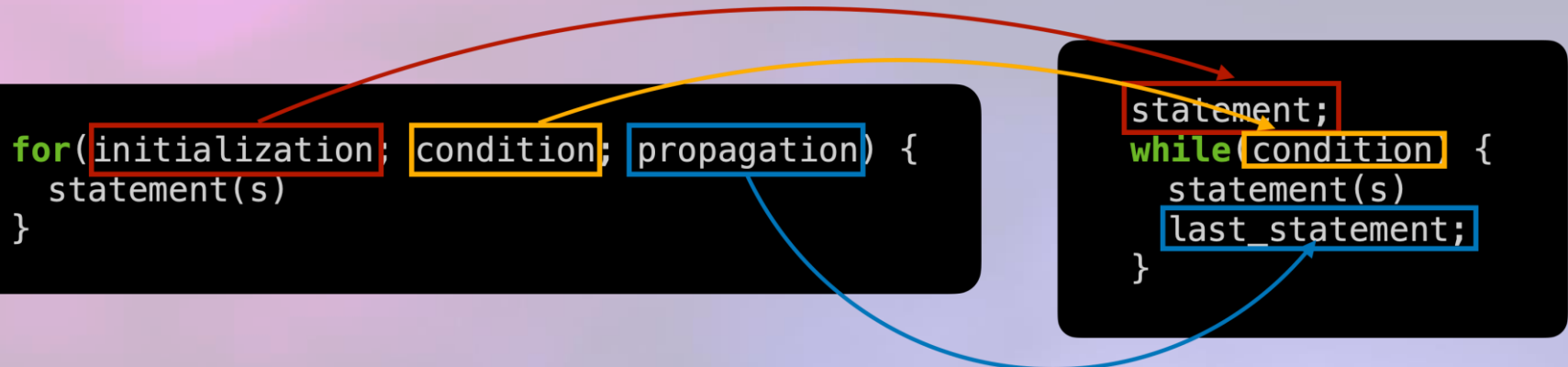
```
while (!has5 && num>0){  
    if (num % 10 == 5)  
        has5 = true;  
    num /= 10;  
}
```

```
System.out.println(num + "has digit 5? " + has5);
```

```
while (!has5 && num>0){  
    if (num % 10 == 5)  
        has5 = true;  
    else  
        has5 = false;  
    num /= 10;  
}  
  
while (!has5 && num>0){  
    has5 = num % 10 == 5;  
    num /= 10;  
}
```



```
for(initialization; condition; propagation) {  
    statement(s)  
}
```



```
statement;  
while(condition) {  
    statement(s)  
    last_statement;  
}
```

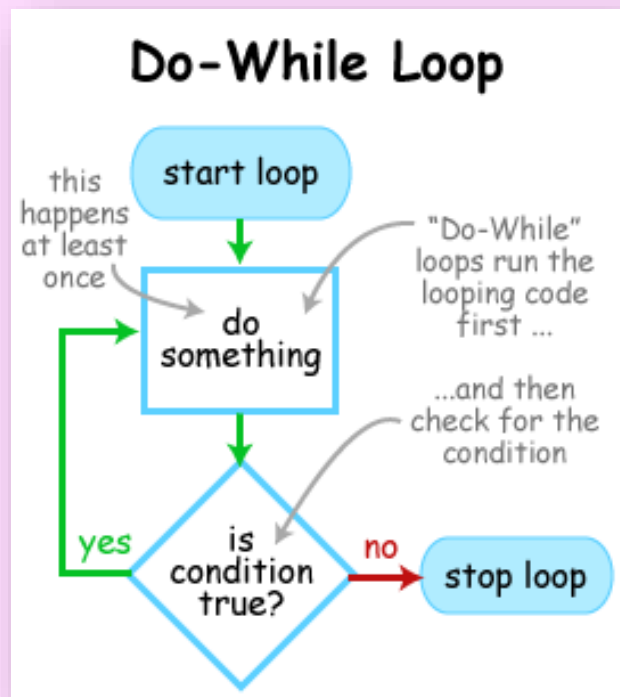
```
while (not edge) {  
  run();  
}
```

```
do {  
  run();  
} while (not edge);
```



לולאת do-while

```
import java.util.Random;
public class Program
{
    public static void main(String[] args)
    {
        int num, count = 0, sum = 0;
        Random rnd = new Random();
        do
        {
            num = rnd.nextInt(100);
            sum = sum + num;
            count++;
        }
        while (sum < 100);
        System.out.format("sum = %d, count = %d \n", sum, count);
    }
}
```

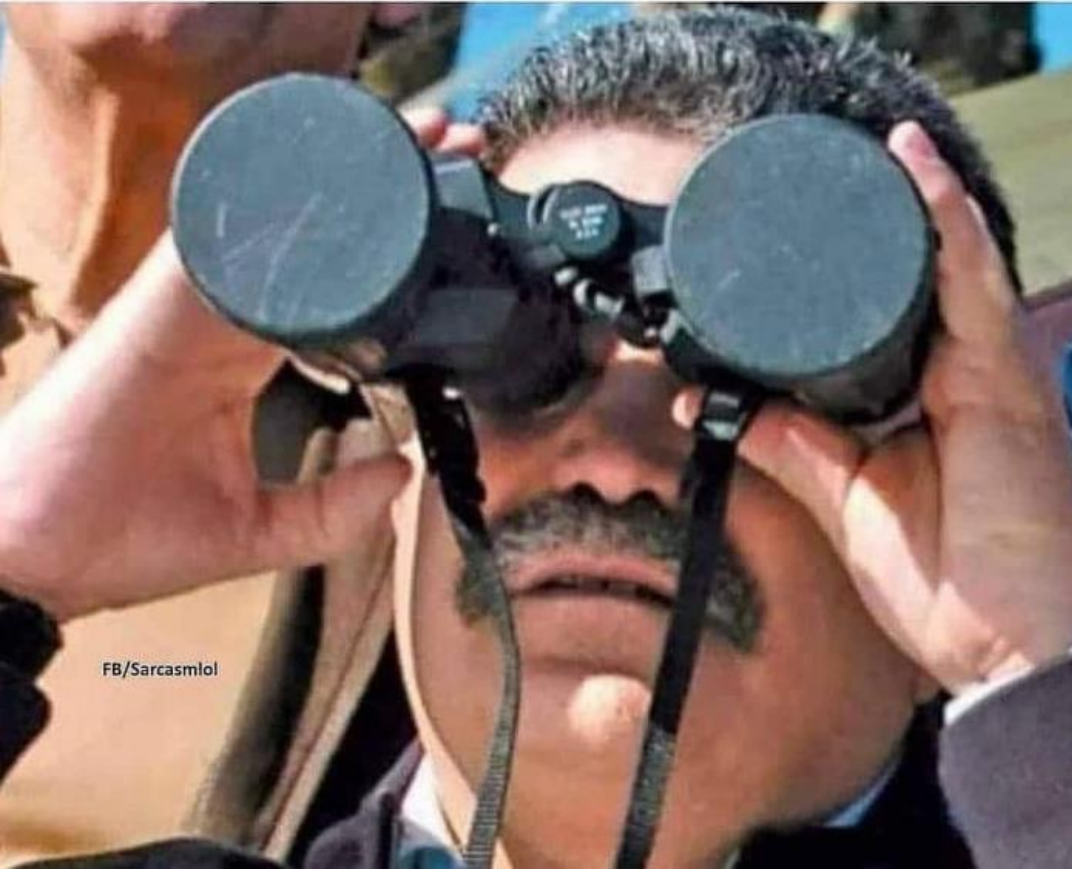


לולאת do-while



```
public static void main(String[] args)
{
    Scanner input = new Scanner(System.in);
    System.out.print("Enter a number : ");
    int num = input.nextInt();
    int dig, revNum = 0;
    do
    {
        dig = num % 10;
        revNum = revNum * 10 + dig;
        num = num / 10;
    }
    while (num > 0);
    System.out.println("The rev num is: " + revNum);
}
```

Me looking at my future



לולאת `for(each)`
בהמשך

🦊 תבנית מניה מותנית 🦊

נקודת מוצא: סדרה של ערכים

מטרה: מספר הערכים בסדרה העונים על תנאי

אלגוריתם:

אתחל את ערך המונה counter

עבור כל אחד מסדרת הערכים

אם הערך עונה על התנאי

הגדל את counter ב-1

החזר counter

🦊 תבנית מניה פשוטה 🦊

נקודת מוצא: סדרה של ערכים

מטרה: מספר הערכים בסדרה

אלגוריתם:

אתחל את ערך המונה counter

עבור כל אחד מסדרת הערכים

הגדל את counter ב-1

החזר counter

שימוש – כמות הופעות של פריט, תמונה או כל מרכיב אחר בסדרת נתונים

בד"כ במניה ערכו של המונה מאותחל ל-0

```
1  import java.util.Random;
2  public class ClassNameHere {
3      public static void main(String[] args) {
4          Random rnd = new Random();
5          int num, count = 0;
6          for(int i = 0; i < 10; i++){
7              num = rnd.nextInt(500);
8              System.out.println(num);
9              if (num % 10 == 0)
10                 count++;
11          }
12          System.out.println(count);
13      }
14  }
15
```

🐱 תבנית צבירה 🐱

נקודת מוצא: סדרה של מספרים

מטרה: סכום המספרים בסדרה

אלגוריתם:

אתחל את ערך הצובר sum

עבור כל אחד מסדרת הערכים

הגדל את sum ב-ערך המספר

החזר sum

שימוש – סכום סדרה מכפלה (עצרה)

בד"כ בסכימה ערכו של המונה מאותחל ל-0. במכפלה ל-1

```
1  import java.util.Random;
2  public class ClassNameHere {
3      public static void main(String[] args) {
4          Random rnd = new Random();
5          int num, sum = 0;
6          for(int i = 0; i < 10; i++){
7              num = rnd.nextInt(500);
8              System.out.println(num);
9              sum += num;
10         }
11         System.out.println(sum);
12     }
13 }
14
```

🐱 תבנית איסוף בקיזוז 🐱

נקודת מוצא: סדרה של מספרים, תנאי

מטרה: האם הסדרה מאוזנת או למי רוב/מיעוט

אלגוריתם:

אתחל את ערך המונה `count` או הצובר `sum`

עבור כל אחד מסדרת הערכים

אם עונה על התנאי

הגדל מונה/צובר

אחרת

הקטן מונה/צובר

החזר תשובה

שימוש – האם סדרה מאוזנת (שליליים/חיוביים)

איזה מועמד מבין שניים זכה

```
1  import java.util.Random;
2  public class ClassNameHere {
3      public static void main(String[] args) {
4          Random rnd = new Random();
5          int num, count = 0;
6          for(int i = 0; i < 10; i++){
7              num = rnd.nextInt(500);
8              System.out.println(num);
9              if (num > 250)
10                 count++;
11             else
12                 count--;
13         }
14         System.out.println(count);
15     }
16 }
17
```

🐱 תבנית מקסימום 🐱

נקודת מוצא: סדרה של ערכים

מטרה: הערך המקסימלי

אלגוריתם:

אתחל את ערך max בערך הראשון או הערך הנמוך ביותר האפשרי

עבור כל אחד מסדרת הערכים

אם הערך גדול מ- max

השם את הערך ב max

החזר max

🐱 תבנית מינימום 🐱

נקודת מוצא: סדרה של ערכים

מטרה: הערך המינימלי

אלגוריתם:

אתחל את ערך min בערך הראשון או הערך הגבוה ביותר האפשרי

עבור כל אחד מסדרת הערכים

אם הערך קטן מ- min

השם את הערך ב min

החזר min

```
1  import java.util.Random;
2  public class ClassNameHere {
3      public static void main(String[] args) {
4          Random rnd = new Random();
5          int num, max = 0;
6          for(int i = 0; i < 10; i++){
7              num = rnd.nextInt(500);
8              System.out.println(num);
9              if (num > max)
10                 max = num;
11          }
12          System.out.println(max);
13      }
14  }
15
```

🐱 תבנית ערך נלווה לערך קיצון 🐱

נקודת מוצא: סדרה של ערכים

מטרה: הערך המקסימלי

אלגוריתם:

אתחל את ערך הקיצון

אתחל את ערך המשתנה הנלווה save (מיקום, שם, הפנייה)

עבור כל אחד מסדרת הערכים

אם הערך הוא ערך קיצון

השם את הערך ב $\max/\min$

השם את הערך הנלווה ב save

החזר save

```
1 import java.util.Random;
2 public class ClassNameHere {
3     public static void main(String[] args) {
4         Random rnd = new Random();
5         int num, max = 0, pos = 0;
6         for(int i = 0; i < 10; i++){
7             num = rnd.nextInt(500);
8             System.out.println(num);
9             if (num > max){
10                 max = num;
11                 pos = i + 1;
12             }
13         }
14         System.out.println(max + " in pos " + pos);
15     }
16 }
17
```

🐱 תבנית שני בגודלו 🐱

נקודת מוצא: סדרה של ערכים

מטרה: הערך השני בגודלו

אלגוריתם:

אתחל את ערך max בערך הראשון או הערך הנמוך ביותר האפשרי

אתחל את ערך sec בערך הראשון או הערך הנמוך ביותר האפשרי

עבור כל אחד מסדרת הערכים

אם הערך גדול מ- max

השם את max ב sec

השם את הערך ב max

אחרת

אם הערך גדול מ- sec

השם את הערך ב- sec

🐱 תבנית שני בקוטנו 🐱

נקודת מוצא: סדרה של ערכים

מטרה: הערך השני בקוטנו

אלגוריתם:

אתחל את ערך min בערך הראשון או הערך הגבוה ביותר האפשרי

אתחל את ערך sec בערך הראשון או הערך הגבוה ביותר האפשרי

עבור כל אחד מסדרת הערכים

אם הערך קטן מ- min

השם את min ב sec

השם את הערך ב min

אחרת

אם הערך גדול מ- sec

השם את הערך ב- sec

```
1 import java.util.Random;
2 public class ClassNameHere {
3     public static void main(String[] args) {
4         Random rnd = new Random();
5         int num, max = 0, sec = 0;
6         for(int i = 0; i < 10; i++){
7             num = rnd.nextInt(500);
8             System.out.println(num);
9             if (num > max){
10                 sec = max;
11                 max = num;
12             }
13             else if(num > sec)
14                 sec = num;
15         }
16         System.out.println(sec);
17     }
18 }
19
20
```

Nested Loops Join

Outer Input



B

A

E

F

D

Inner Input



A

B

E

B

C

```
for(int i = 12; i <= 14; i++) { 
    for(int j = 1; j <= 10; j++) {
        System.out.println(i * j);
    }
}
```

[illegible]

```

    for(int i = 1 ; i <= 4; i++){
        for(int j = 1 ; j <= 3; j++){
            System.out.print('*');
        }
        System.out.println();
    }

```

i	j
1	1
	2
	3
2	1
	2
	3
3	1
	2
	3
4	1
	2
	3
5	

```

* * *
* * *
* * *
* * *

```

```

for(int i = 4 ; i > 0; i--){
    int j = i;
    while (j > 0){
        System.out.print('*');
        j--;
    }
    System.out.println();
}

```

i	j
4	4
	3
	2
	1
	0
3	3
	2
	1
	0
2	2
	1
	0
1	1
	0
0	

```

* * * *
* * *
* *
*

```



פעולות

הצהרה שהתוכנית תעשה שימוש במרחבי השימוש המפורטים.

מרחב שמות – namespace

אוסף של מחלקות שנאספו יחד וקיבלו שם משותף

```
import java.util.Scanner;  
import java.util.Random;
```

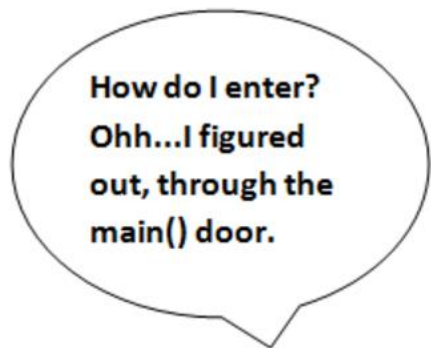
```
public class Program {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
    }
```

פעולה – אוסף של הוראות לביצוע, המאוגדות יחד, שניתן להן שם.

המחלקה – אוסף של פעולות (ותכונות)

הפעולה הראשית - main



פרוייקט ב-Java מכיל
מחלקה ראשית.
במחלקת הראשית קיימת
פעולה ראשית, ה-main.
הפעולה main חייבת
להופיע במחלקה הראשית
של התוכנית שלנו ואין
לשנות את צורת הכתיבה
שלה.
הסיבה: הרצת התוכנית
מתחילה תמיד מהפעולה
הראשית, ה-main.
צרכי המשימה יבואו לידי
ביטוי בפעולה הראשית, ה-main,
והיא אשר תקבע
את סדר ביצוע הפעולות
בהתאם לנדרש.

סוגי פעולות

ישנם שני סוגים של פעולות:

- פעולות המחזירות ערך
- פעולות שאינן מחזירות ערך

```
public static double avg(int n1, int n2) {
```

.....

```
return a;
```

```
}
```

```
public static void printAvg(int n1, int n2) {
```

.....

```
}
```

פעולה המחזירה ערך

בכותרת נצהיר על טיפוס

הנתונים המוחזר.

בפעולה עצמה תופיע ההוראה

return

פעולה שאינה מחזירה ערך.

בכותרת נצהיר כי הפעולה אינה

מחזירה ערך, על ידי כתיבת

void המילה השמורה

כותרת הפעולה (חתימה)

כל טיפוס חוקי בשפה - פשוט

int, double, boolean

או ממחלקה

String, Bucker, Student, Turtle

אם הפעולה אינה מחזירה ערך - void

אם הפעולה אינה
מופעלת על עצם, יש
להגדירה כסטטית

למי מותר להשתמש?
פומבית public –
לכולם
פרטית private – רק
במחלקה שבה כתובה



public static double avg(int count)

{

}

שם הפעולה – יתחיל באות
קטנה,

במקרה ומורכב ממספר מילים:
findMaxAvg



רשימת פרמטרים מופרדים בפסיקים:
לכל פרמטר: הצהרת הטיפוס + שם:

avg()

avg(int count)

avg(int num1, double num2)

signature

*return
type*

*method
name*

*argument
type*

*argument
variable*

```
public static double harmonic ( int n )
```

```
{
```

*local
variable*

```
double sum = 0.0;
```

*method
body*

```
for (int i = 1; i <= n; i++);
```

```
    sum += 1.0/i;
```

```
return sum;
```

```
}
```

return statement

זימון של פעולה



- זימון של פעולה הוא קריאה לביצוע הפעולה מתוך הפעולה הראשית (או מפעולה אחרת)
- הוראת הזימון היא כתיבת שם הפעולה ורשימת הערכים המועברים לפעולה (הפרמטרים)
- הערכים חייבים להתאים לטיפוסים שהוגדרו בכותרת הפעולה.
- **כאשר הפעולה מחזירה ערך** צריך לדעת לטפל ("לקבל") את הערך המוחזר
- **פעולה שלא זומנה, לא מתבצעת**

הפרמטרים הם משתנים פנימיים של הפעולה, המאותחלים בעת הזימון לערכים המועברים (ארגומטים). הם מוכרים רק בפעולה עצמה.

```
// method definition
public static void printArea(int x, int y) {
    System.out.println(x * y);
}

public static void main(String[] args) {
    // calling method
    printArea(2, 4);
}
```

Parameters

Arguments

הארגומנטים חייבים להיות עם ערך בעת הזימון – הם יכולים להיות קבועים – 5, 4.3, "Happy", 'a', ויכולים להיות משתנה שיש לו ערך או ביטוי הניתן לחישוב (לרבות זימון פעולה המחזירה ערך או פעולה בונה new Random())

```
class Test {  
    // method definition  
    public static int getArea(int x, int y) {  
        int z = x * y;  
        return z;  
    }  
    public static void main(String[] args) {  
        // calling method  
        int area = getArea(2, 4);  
        System.out.println("Area: " + area);  
    }  
}
```



area = getArea(2,4)
equivalent to
area = 8

```

public class ClassA{

    public static void main(String []args){
        printTwice("ab");
        printTwice('a');
        ClassA.printTwice("ab");
        ClassA.printTwice('a');

        ClassB.printTwice("ab");
        ClassB.printTwice('a');
    }

    public static void printTwice(String s){
        System.out.println("1" + s);
        System.out.println("1" + s);
    }

    public static void printTwice(char s){
        System.out.println("1" + s);
        System.out.println("1" + s);
    }
}

class ClassB{

    public static void printTwice(String s){
        System.out.println("2" + s);
        System.out.println("2" + s);
    }

    public static void printTwice(char s){
        System.out.println("2" + s);
        System.out.println("2" + s);
    }
}

```

זימון ממחלקה אחרת



העמסה overload



1ab
 1ab
 1a
 1a
 1ab
 1ab
 1a
 1a
 2ab
 2ab
 2a
 2a

```

public class ClassA{
    public static void main(String []args){
        System.out.println(sum(1,2));
        System.out.println(ClassB.sum(1,2,3,4));

        int x = ClassB.sum(5,6,sum(1,2));

        if (ClassB.sum(1,2,3,4) < x)
            System.out.println(ClassB.sum(9,1,1));
    }

    public static int sum(int x1, int x2){
        return x1+x2;
    }
}

class ClassB{
    public static int sum(int x1, int x2, int x3){
        return x1+ClassA.sum(x2,x3);
    }

    public static int sum(int x1, int x2, int x3, int x4){
        return ClassA.sum(x1,x2) + ClassA.sum(x3,x4);
    }
}

```

אם הפעולה מחזירה
ערך – נעשה איתו
משהו



3
10
11

פעולה הקוראת
לפעולה



דוגמה	דרך
<code>double mem = average3(grade1,grade2,grade3);</code>	השמה
<code>System.out.println(average3(grade1,grade2,grade3);</code>	פלט
<code>if(average3(grade4,grade5,grade6)> average3(grade1,grade2,grade3)) System.out.print ("very well");</code>	השוואה
<code>double avg = average3(grade1,grade2,grade3)+10.0;</code>	חישוב
<code>average3(grade1,grade2,grade3);</code>	?

תיעוד הפעולה – מקובל לתעד כל פעולה בתכנית. (תכנית הלימודים מחייבת תיעוד). התיעוד יכול: תיאור של מה שהפעולה מבצעת ומה היא מחזירה. אם הפעולה מקבלת ערכים (פרמטרים), נרשום תיאור של הערכים האלו. אם על הערכים האלה לקיים תנאי כלשהו כדי שהפעולה תפעל כראוי, נרשום תנאים אלו כהנחות שאנו מניחים לגביהם (למשל: הנחה: המספר אינו שלילי).

דוגמאות לתיעוד:

- פעולה המקבלת ... (תיאור הפרמטרים) ... ומחזירה ... (מה מחזירה הפעולה) ... הנחות: ... (רשימת ההנחות שמניחה הפעולה לגבי הפרמטרים) ...
- פעולה המקבלת ... (תיאור הפרמטרים) ... ומבצעת ... (מה מבצעת הפעולה) ... הנחות: ... (רשימת ההנחות שמניחה הפעולה לגבי הפרמטרים) ...
- טענת כניסה: ... (אילו פרמטרים מקבלת הפעולה ומהן ההנחות לגביהם) טענת יציאה: ... (מה עושה / מחזירה הפעולה)

נוסח שאלות



כתבו זימון



כתבו קטע קוד



כתבו פעולה



כתבו תכנית